# Dynamic Server Scaling: Performance and Cost Trade-Offs of Vertical, Horizontal, and Hybrid Autoscaling Under Variable Workloads

**Shucayb Said**

Department of Engineering Science, University West (Högskolan Väst), Trollhättan, Sweden — Course: DTA400 (Autumn 2026)

*(Simulation software and experimental suite developed jointly with Firat Kaya; individual report written by Shucayb Said.)*

***Abstract*—Cloud-hosted web applications experience pronounced diurnal variations and sudden traffic spikes that make static server provisioning either prohibitively expensive or vulnerable to severe queue saturation. While reactive autoscaling mitigates this tension, cloud architects must choose between vertical scaling (resizing per-instance capacity), horizontal scaling (adding or removing server instances), and hybrid scaling (combining both dimensions). This paper presents a reproducible discrete-event simulation study built in Python and SimPy that evaluates vertical, horizontal, and two hybrid scaling strategies against single-server and multi-server static baselines across six stochastic traffic scenarios. The simulator models heterogeneous request classes (small, medium, heavy), centralized FIFO queuing, instance lifecycle states (STOPPED, BOOTING, ACTIVE, DRAINING, STANDBY), provisioning delays, post-horizon queue draining, and continuous-time normalized operating costs across N = 10 common-random-number seeds. Paired statistical evaluations show that under a standard service-level objective (P95 ≤ 1.0 s), horizontal scaling achieves the lowest operating cost on variable baseline traffic (1.9789 units, -6.78% vs. vertical and -6.48% vs. 3-tier hybrid, both statistically significant at the 95% level) due to finer 1-unit capacity increments at intermediate loads. However, horizontal scaling suffers severe P95 latency degradation when cold-start delays reach 20–30 s (P95 = 3.78 s). Maintaining 1 warm standby instance (2 s resume at 25% hourly cost) reduces horizontal P95 latency by 79.4% under recurring spikes, while a 60 s scale-down observation window prevents premature deprovisioning across 45 s inter-spike lulls. Finally, under a strict tail-latency target (P95 ≤ 0.35 s) or per-instance vertical capacity limits (c_max,node = 2), horizontal scaling fails in 10/10 runs, whereas 3-tier hybrid scaling satisfies P95 ≤ 0.35 s in 10/10 runs (and 4-tier adaptive hybrid in 8/10 runs at -4.96% lower mean cost).

***Keywords*—*Autoscaling, cloud computing, discrete-event simulation, horizontal scaling, hybrid scaling, queueing systems, SimPy, vertical scaling.***

## I. INTRODUCTION

Modern web applications face non-stationary arrival processes characterized by multi-fold shifts between off-peak lulls, sustained peak hours, and abrupt flash crowds [1], [2]. Provisioning fixed capacity for peak demand (static_4) guarantees low response times but wastes over 55% of paid capacity during normal operation, whereas provisioning for average demand (static_1 or static_2) causes catastrophic queue growth during traffic spikes [3]. Dynamic resource scaling resolves this dilemma by adjusting provisioned capacity in response to observed utilization and queue backlog.

Cloud infrastructure providers support three primary elasticity paradigms:

1. **Vertical scaling (scale-up/down):** Dynamically adjusting the CPU quota or core allocation (c) of an existing server instance [4]. Higher per-instance capacity accelerates individual request service times (t_s = W/c) but is bounded by single-host hardware ceilings and coarse binary sizing steps (1 → 2 → 4).
2. **Horizontal scaling (scale-out/in):** Adding or terminating parallel server instances (k) of fixed unit capacity [5]. Horizontal scaling offers linear unit-by-unit granularity (1 → 2 → 3 → 4) and unbounded cluster size, but incurs instance cold-start delays (10–30 s) and cannot reduce the single-thread execution time of heavy requests.
3. **Hybrid scaling:** Combining vertical resizing for fast initial capacity boosts and lower per-request service time with horizontal scale-out to surpass single-node ceilings [6].

Despite extensive literature on reactive and predictive autoscalers [1], [6], direct quantitative comparisons that isolate the structural trade-offs between vertical, horizontal, and hybrid scaling under identical arrival streams, normalized capacity bounds, and continuous-time cost accounting remain scarce. To address this gap, this study investigates three formal research questions:

- **RQ1:** *Which scaling strategy achieves the lowest estimated operating cost while satisfying a defined response-time objective (P95 ≤ 1.0 s)?*
- **RQ2:** *How do startup delays (0–30 s), warm standby operation, and delayed shutdown observation windows (30–120 s) affect server cost and response time during traffic spikes?*
- **RQ3:** *Under which workload, latency-SLO, and hardware-constraint conditions is hybrid scaling more cost-effective than either vertical or horizontal scaling alone?*

## II. BACKGROUND AND RELATED WORK

### A. Cloud Elasticity and Queuing Dynamics

Lorido-Botran et al. [1] and Qu et al. [5] classify cloud autoscalers into reactive threshold rules, queuing models, control theory, and time-series forecasters. Threshold-based reactive controllers remain the industry standard in Kubernetes (HPA/VPA) and AWS Auto Scaling due to their simplicity and model-free operation [6]. From a queuing perspective [3], [7], a single vertically scaled server of capacity c (M/G/1-speed-c) and a horizontal pool of k=c unit-capacity servers (M/G/k-speed-1) share the same maximum throughput $\mu_{total} = c / E[W]$ and stability condition $\rho = \lambda E[W] / c < 1$. However, at low-to-moderate utilization, the single fast server yields lower mean response time because service time itself shrinks (W/c), whereas at intermediate loads, multi-server unit granularity avoids binary over-provisioning.

### B. Cold Starts, Warm Pooling, and Scale-Down Hysteresis

Provisioning new virtual machines or containers introduces a non-zero startup latency ($\tau_{boot}$) during which arriving traffic accumulates in the waiting queue [8], [9]. Keeping pre-initialized warm standby instances reduces transition latency to a brief resume delay ($\tau_{warm} << \tau_{boot}$) in exchange for a continuous holding fee [5]. Conversely, when traffic subsides, immediate termination risks thrashing if a second spike arrives shortly thereafter; autoscalers therefore enforce a scale-down observation window ($\tau_{down}$) and post-action cooldown ($\tau_{cool}$) [1], [6]. While trace analyses of production clusters such as Google Borg [2] document macroscopic job resource usage, controlled discrete-event simulation (DES) using process-oriented frameworks like SimPy [10] is required to isolate request-level tail latency and exact capacity-time integrals across counterfactual scaling policies.

## III. METHODOLOGY AND SIMULATION DESIGN

### A. Discrete-Event Simulation Architecture

We implemented a modular discrete-event simulator in Python 3 using SimPy 4.1.2 [10]. The architecture comprises five decoupled components:

1. **Stochastic Workload Generator (`workload.py`):** Generates request arrivals via a non-homogeneous Poisson process with piecewise-constant rate $\lambda(t)$ over a $T_{sim} = 3600$ s arrival horizon. Inter-arrival sampling is strictly truncated at segment boundaries using the memoryless property of the exponential distribution so rate jumps take effect instantaneously. Each request is assigned one of three CPU work classes: small (p=0.50, W=0.05 CPU-s), medium (p=0.35, W=0.15 CPU-s), or heavy (p=0.15, W=0.35 CPU-s),

yielding mean work E[W] = 0.13 CPU-s (1 capacity unit ≈ 7.69 req/s maximum throughput).

2. **FIFO Request Queue (`server.py`):** Holds waiting requests and integrates exact continuous-time queue area $\int Q(t)\,dt$ at every enqueue/dequeue event.

3. **Server Pool and Lifecycle State Machine (`server.py`):** Manages ServerInstance objects across five states: STOPPED, BOOTING, ACTIVE, DRAINING, and STANDBY. An ACTIVE server of capacity c processes one request at a time in $t_s = W/c$ s. Synchronous dispatch locking (server.busy = True prior to SimPy process scheduling) prevents simultaneous timestamp collisions. When an active server is selected for scale-down while busy, it transitions to DRAINING, finishes its in-flight request without interruption, and only then enters STANDBY or STOPPED (zero dropped requests).

4. **Reactive Autoscaler (`scaling.py`):** Samples rolling utilization and queue length every $\Delta t_{mon} = 5.0$ s. Scale-up triggers when rolling utilization $\rho_{hat} \geq 0.85$ or when waiting queue length $Q(t) \geq 2$ for 2 consecutive samples (10.0 s). Scale-down triggers when rolling utilization over the observation window ($\tau_{down} = 60.0$ s) satisfies $\rho_{avg,60s} \leq 0.35$ and $Q(t) = 0$. A cooldown of $\tau_{cool} = 60.0$ s follows each completed scaling transition.

5. **Continuous-Time Metrics and Cost Collector (`metrics.py`):** Integrates exact capacity-seconds and busy CPU-seconds across every state change. The complete simulation source code, test suite, and experimental datasets are publicly available at https://github.com/SAS2005/Project-DTA400.

### *B. Post-Horizon Drain Phase and Throughput Accounting*

Request arrivals occur strictly within [0, T_sim) (T_sim = 3600 s). To ensure conservation of work (completed_requests == generated_requests), any requests remaining in the queue or in flight at t = 3600 s are processed to completion during a post-arrival drain phase of duration T_drain = max(0, t_last_completion - T_sim), giving total elapsed time T_elapsed = T_sim + T_drain. Because saturated configurations (static_1) accumulate backlogs that drain for T_drain in [0.8, 29.7] s after T_sim, dividing total completions by T_sim would misstate active processing rate. We therefore report both **arrival-window throughput** ($N_{comp}(t \leq T_{sim}) / T_{sim}$) and **end-to-end completion rate** ($N_{comp} / T_{elapsed}$), and integrate server operating costs over the full T_elapsed.

### *C. Scaling Strategies, Static Baselines, and Cost Model*

All dynamic strategies operate within identical capacity bounds (C_min = 1 unit, C_max = 4 units) and share identical controller thresholds (Table I). We also distinguish single-server vertical static baselines (static_1 1× 1, static_2 1× 2, static_4 1× 4) from multi-server horizontal static baselines (static_horizontal_2 2× 1, static_horizontal_4 4× 1).

**TABLE I: Scaling Strategy Configurations and Baseline Topologies**

| Strategy ID | Topology (k × c) | Cap. (C) | Transition Delay |
|---|---|---|---|
| static_1 | 1 server (1× 1) | 1.0 | Fixed (none) |
| static_2 (_vert_2) | 1 server (1× 2) | 2.0 | Fixed (none) |
| static_4 (_vert_4) | 1 server (1× 4) | 4.0 | Fixed (none) |
| static_horiz_2 / _4 | 2 or 4 servers (k× 1) | 2.0, 4.0 | Fixed (none) |
| vertical | 1 server: [1]↔[2]↔[4] | 1.0... 4.0 | $\tau_{vert} = 5.0$ s |
| horizontal | 1× 1 ↔ 2× 1 ↔ 3× 1 ↔ 4× 1 | 1.0... 4.0 | $\tau_{boot} = 10$ s (2 s warm) |
| hybrid (3-tier) | 1× 1 ↔ 1× 2 ↔ 2× 2 | 1, 2, 4 | 5 s vert. / 10 s horiz. |
| hybrid_adaptive | [1]↔[2]↔[2,1]↔[2,2] | 1, 2, 3, 4 | 5 s vert. / 10 s horiz. |

Total normalized operating cost J_total (where 1 active capacity unit for 1 hour = 1.00 cost unit) is integrated over [0, T_elapsed]:

$$J_{total} = (1 / 3600)\,[\,\beta_{act} \int C_{act}(t)\,dt + \beta_{prov} \int C_{prov}(t)\,dt + \beta_{sb} \int C_{sb}(t)\,dt\,]$$

with $\beta_{act} = 1.00$, $\beta_{prov} = 1.00$, and $\beta_{sb} = 0.25$ units/(cap·hr).

### *D. Experimental Scenarios and Statistical Protocol*

Six 1-hour scenarios are evaluated: (1) variable_baseline (0–600 s at 3 req/s, 600–1500 s at 6 req/s, 1500–2100 s at 12 req/s, 2100–2220 s spike at 20 req/s, 2220–3000 s at 6 req/s, and 3000–3600 s at 3 req/s), (2) low_steady (3 req/s), (3) normal_steady (6.5 req/s), (4) high_steady (13 req/s), (5) sudden_spike (6 req/s baseline with a 180 s burst to 22 req/s), and (6) recurring_spikes (five bursts of 20 req/s separated by 45 s lulls at 4 req/s). Every configuration is replicated across N = 10 independent seeds (42..51) using Common Random Numbers (clone_workload()). For any two strategies A and B, we compute seed-by-seed paired differences $d_i = x_{A,i} - x_{B,i}$ and the two-sided 95% Student-t confidence interval $\Delta d \pm t_{(0.975,9)}\, s_d / \sqrt{10}$ ($t_{(0.975,9)} = 2.262$).

## IV. EXPERIMENTAL RESULTS

### *A. Baseline Strategy Comparison (RQ1)*

Table II summarizes Experiment 1 on variable_baseline (~ 23,270 requests/run, theoretical arrival rate 6.464 req/s). Single-server static_1 (1× 1) and static_2 (1× 2) cannot sustain the high (12 req/s) and spike (20 req/s) segments, suffering severe queue saturation with P95 latencies of 499.19 s and 21.43 s (Fig. 1). Moreover, static_1 requires an average post-horizon drain phase of T_drain = 5.27 s (up to 29.69 s), yielding an arrival-window throughput of 6.4525 req/s versus an end-to-end completion rate of 6.4548 req/s (83.9% utilization) and incurring 1.0015 cost units due to post-horizon server operation. All other strategies complete 100% of requests within T_sim (T_drain = 0 s, throughput 6.4642 req/s). static_4 (1× 4) achieves P95 = 0.1212 s (10/10 runs compliant) but costs 4.0000 units at only 21.0% utilization.

**TABLE II: Baseline Performance and Cost on variable_baseline (N=10 Seeds, Mean ± 95% CI)**

| Strategy | Avg RT (s) | P95 RT (s) | Total Cost (units) | SLO ≤ 1.0 s | Strict ≤ 0.35 s |
|---|---|---|---|---|---|
| static_1 (1× 1) | 199.83 ± 6.04 | 499.19 ± 10.49 | 1.0015 ± 0.0021 | 0/10 (0%) | 0/10 (0%) |
| static_2 (1× 2) | 2.3865 ± 0.1919 | 21.43 ± 1.63 | 2.0000 ± 0.0000 | 0/10 (0%) | 0/10 (0%) |
| static_4 (1× 4) | 0.0462 ± 0.0006 | 0.1212 ± 0.0019 | 4.0000 ± 0.0000 | 10/10 (100%) | 10/10 (100%) |
| vertical (1× 1..4) | 0.0982 ± 0.0012 | 0.3256 ± 0.0060 | 2.1229 ± 0.0111 | 10/10 (100%) | 10/10 (100%) |
| horizontal (1..4× 1) | 0.1629 ± 0.0022 | 0.3907 ± 0.0065 | **1.9789 ± 0.0242** | 10/10 (100%) | 0/10 (0%) |
| hybrid (1× 1→ 2× 2) | 0.1090 ± 0.0010 | 0.3321 ± 0.0056 | 2.1160 ± 0.0122 | 10/10 (100%) | 10/10 (100%) |
| hybrid_adaptive | 0.1191 ± 0.0038 | 0.3467 ± 0.0051 | 2.0110 ± 0.0560 | 10/10 (100%) | **8/10 (80%)** |

All four dynamic strategies satisfy the default SLO (P95 ≤ 1.0 s) in **10/10 runs (100%)** while reducing cost by 46.9%–50.5% relative to static_4 (39.7%–42.7% utilization). Answering **RQ1**, **horizontal scaling achieves the lowest operating cost (1.9789 ± 0.0242 units, 0.0850 units/1k req)**. Paired seed-by-seed comparisons (Table III) confirm that horizontal scaling is significantly cheaper than both vertical ($\Delta d$ = -0.1440 units, 95% CI [-0.1730, -0.1151], -6.78%) and 3-tier hybrid ($\Delta d$ = -0.1371 units, 95% CI [-0.1668, -0.1074], -6.48%). During the 1500–2100 s high-load segment ($\lambda$ = 12 req/s, mean demand 1.56 units), horizontal provisions an intermediate **3-unit state (3× 1)**, whereas binary vertical and 3-tier hybrid jump from 2 to 4 units. Conversely, vertical and hybrid achieve significantly lower P95 latency (0.3256 s and 0.3321 s vs. 0.3907 s; $\Delta d_{P95}$ = -0.0650 s and -0.0586 s) because servers with $c \geq 2$ cut individual job service times in half (Fig. 2).

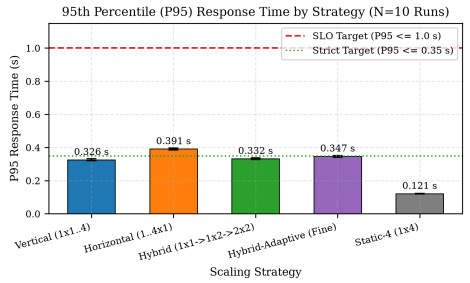



Fig. 1. P95 response time across scaling strategies on variable_baseline (N=10 seeds, Mean +/- 95% CI).

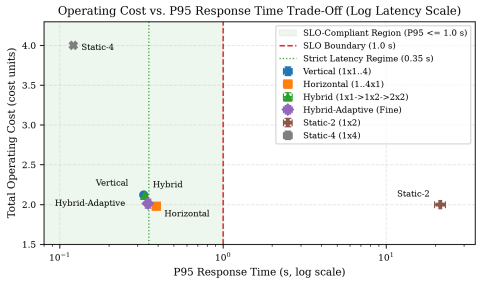



Fig. 2. Pareto trade-off between normalized operating cost and P95 response time on variable_baseline.

### *B. Cold-Start Delay, Warm Standby, and Shutdown Sensitivity (RQ2)*

1. **Startup Delay Sensitivity (0–30 s, Fig. 3):** Under sudden_spike (6 → 22 req/s), horizontal satisfies P95 ≤ 1.0 s in 10/10 runs for τ_boot ≤ 10 s (P95 = 0.4912 s at 10 s), but degrades to P95 = 1.1506 s (1/10 runs compliant) at τ_boot = 20 s and **P95 = 3.7764 s (0/10 runs compliant) at τ_boot = 30 s**. Because horizontal scales out sequentially one unit at a time (2 → 3 → 4) with a cooldown between steps, a 30 s boot delay leaves capacity below spike demand (2.86 units) for over a minute. In contrast, vertical and hybrid jump directly from 2 → 4 units in a single transition, maintaining P95 = 0.9032 s (9/10 runs compliant) even at τ = 30 s.

2. **Warm Standby Operation (Fig. 4a):** Under recurring_spikes with τ_boot = 30 s, cold-only horizontal and hybrid suffer severe P95 violations (2.5964 s and 6.4757 s, 0/10 runs compliant). Maintaining 1 warm standby instance (τ_warm = 2 s, β_sb = 0.25) reduces horizontal P95 by **79.4%** to 0.5342 s (restoring **10/10 SLO compliance**) for an 11.9% cost increase (2.0209 → 2.2611 units), and cuts hybrid P95 by **66.9%** (6.4757 → 2.1426 s). On predictable traffic (variable_baseline), warm standby yields zero latency reduction while adding 8.9%–10.5% holding cost.

3. **Shutdown Observation Delay (30–120 s, Fig. 4b):** Under recurring_spikes (five bursts separated by 45 s lulls), a short shutdown window (τ_down = 30 s) triggers scale-down during every lull, forcing repeated cold re-boots on the next burst (P95 = 1.3563 s for vertical, 2.6285 s for horizontal, and 5.1286 s for hybrid; 0/10 runs compliant). Extending τ_down to **60 s** bridges the 45 s lulls, slashing P95 latency by 72%–90% to 0.3775–0.5177 s (10/10 runs compliant) for only +4.0%–6.3% cost. Increasing τ_down to 120 s adds another +5.0%–5.5% cost with negligible latency gain (< 0.02 s).

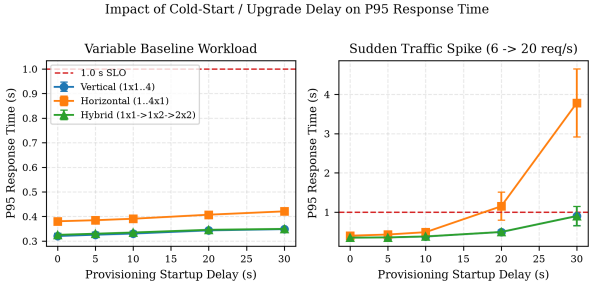



Fig. 3. Impact of provisioning startup delay (0-30 s) on P95 latency under variable_baseline and sudden_spike.

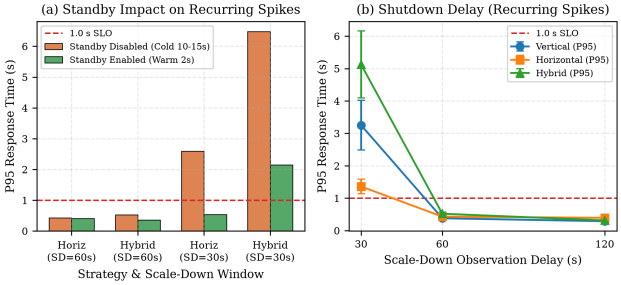



Fig. 4. (a) Warm standby mitigation under 30-s cold starts; (b) Shutdown observation window sensitivity under recurring_spikes.

### *C. Multi-Scenario Analysis and Hybrid Scaling Regimes (RQ3)*

Table III presents the paired seed-by-seed statistical comparisons (d_i = x_A,i − x_B,i) across key regimes to answer **RQ3**:

**TABLE III: Paired Seed-by-Seed Statistical Comparisons (N=10 Seeds, Common Random Numbers)**

| Scenario & Metric | Pair (A - B) | Mean Diff (Δd) | 95% CI of Diff | Rel. % | Sig. (95%)? |
|---|---|---|---|---|---|
| variable_baseline Cost | horiz - vert | -0.1440 u | [-0.1730, -0.1151] | -6.78 % | **Yes** |
| variable_baseline Cost | horiz - hybrid | -0.1371 u | [-0.1668, -0.1074] | -6.48 % | **Yes** |
| variable_baseline Cost | horiz - hyb_adap | -0.0321 u | [-0.0730, +0.0089] | -1.60 % | No (Tied) |
| variable_baseline Cost | hyb_adap - hybrid | -0.1050 u | [-0.1630, -0.0470] | -4.96 % | **Yes** |
| variable_baseline Cost | hybrid - vert | -0.0069 u | [-0.0146, +0.0007] | -0.33 % | No (Tied) |
| variable_baseline P95 | hybrid - horiz | -0.0586 s | [-0.0659, -0.0512] | -14.99 % | **Yes** |
| sudden_spike Cost | hybrid - horiz | -0.0258 u | [-0.0408, -0.0108] | -1.40 % | **Yes** |
| sudden_spike Cost | vert - horiz | -0.0222 u | [-0.0390, -0.0054] | -1.20 % | **Yes** |
| sudden_spike Cost | hybrid - vert | -0.0036 u | [-0.0084, +0.0012] | -0.20 % | No (Tied) |

1. **Strict Latency SLOs (P95 ≤ 0.35 s) — Mean Feasibility vs. Per-Run Compliance:** On variable_baseline, horizontal fails P95 ≤ 0.35 s in **0/10 runs** (P95 = 0.3907 s, min seed 0.3770 s) because a capacity-1 server requires 0.35 s of pure execution time just to serve a heavy request (0.35 CPU-s). Evaluating the remaining strategies requires distinguishing *mean-P95 feasibility* from *strict per-run compliance*:

- Under a **mean-P95 criterion (mean-P95 ≤ 0.35 s)**, hybrid_adaptive (2.0110 units, mean-P95 = 0.3467 s) is the lowest-cost feasible strategy, saving 4.96% (-0.1050 units, statistically significant) relative to 3-tier hybrid (2.1160 units) and 5.27% relative to vertical (2.1229 units). However, because hybrid_adaptive includes a capacity-1 server in its 3-unit state ([2,1]), 2 of 10 seeds marginally exceed 0.35 s (max = 0.35001 s), yielding **8/10 (80%) per-run compliance**.

- Under **strict 100% per-run compliance (10/10 seeds satisfying P95_i ≤ 0.35 s)**, only 3-tier hybrid (mean-P95 = 0.3321 s, max = 0.3469 s) and vertical (mean-P95 = 0.3256 s, max = 0.3405 s) qualify. While hybrid has a slightly lower sample mean cost than vertical (2.1160 vs. 2.1229 units), the paired difference Δd = -0.00694 units (s_d = 0.01074) has a 95% CI of [-0.01462, +0.00074] that includes zero, indicating statistical parity in cost.

2. **Sudden Traffic Spikes (`sudden_spike`):** Both hybrid (1.8185 units) and vertical (1.8221 units) are **statistically significantly cheaper than `horizontal` (1.8443 units)** (Δd_hyb-hor = -0.02582 units, s_d = 0.02100, 95% CI [-0.04084, -0.01079], -1.40%) while achieving 23.1% lower P95 latency (0.3779 s vs. 0.4912 s), because they deprovision from 4 → 2 units in a single step after the spike rather than stepping through 3 units. However, the -0.00361 unit (-0.20%, s_d = 0.00668) difference between hybrid and vertical has a

95% CI of [-0.00839, +0.00117] that includes zero; thus, hybrid and vertical are statistically tied in cost on sudden_spike.

3. **Per-Node Vertical Hardware Ceiling (c_max,node = 2):** In cloud environments where a single instance cannot scale vertically beyond c = 2 (due to NUMA bounds, container quotas, or single-thread limits), pure vertical scaling caps at static_2 (1× 2) and fails with P95 = 21.43 s (0/10 runs). Under this constraint, **hybrid scaling is uniquely superior**: hybrid (2× 2) satisfies both P95 ≤ 1.0 s and P95 ≤ 0.35 s in **10/10 runs (100%)** at 2.1160 units, while hybrid_adaptive ([2,1] → [2,2]) satisfies P95 ≤ 1.0 s in **10/10 runs** at 2.0110 units.

## V. DISCUSSION AND THREATS TO VALIDITY

Our experimental findings demonstrate that autoscaling performance is governed by the interaction between **capacity step granularity** and **per-server service speedup**. Horizontal scaling (1..4x1) excels in cost efficiency when traffic dwells at intermediate demand (λE[W] ≈ 1.56 units) and the latency SLO is relaxed (P95 ≤ 1.0 s), because its 1-unit steps avoid over-provisioning. Conversely, when single-request execution time dominates tail latency (P95 ≤ 0.35 s) or cold starts are slow (τ_boot ≥ 20 s), vertical and hybrid scaling dominate because c=2 instances halve request service times and 2-unit jumps halve the number of scaling cycles required to reach peak capacity. Furthermore, paired statistical analysis is essential when comparing strategies with overlapping capacities: small sample-mean differences between hybrid and vertical (0.20%–0.33%) have 95% confidence intervals spanning zero, showing that hybrid scaling's primary advantage over pure vertical scaling lies not in raw cost under unconstrained vertical resizing, but in achieving identical fast-server latency while respecting per-node vertical hardware ceilings (c_max,node = 2).

**Limitations and Threats to Validity:** (1) *Idealized service speedup:* Our model assumes t_s = W/c; in practice, non-CPU I/O waits and Amdahl's law limit speedups at c=4, which further favors hybrid (2× 2) over vertical (1× 4). (2) *Live vertical resizing:* We modeled in-place container CPU quota updates (τ_vert = 5 s) without connection resets; hypervisors requiring full VM reboots would incur service interruption. (3) *Synthetic Poisson arrivals and modeled costs:* While seeded non-homogeneous Poisson profiles and normalized linear capacity costs enable controlled Common Random Numbers comparisons, real production clouds exhibit sub-second burstiness and tiered instance pricing.

## VI. CONCLUSION

This study evaluated vertical, horizontal, and hybrid server autoscaling using a validated Python/SimPy discrete-event simulation across 10 paired random seeds. Regarding **RQ1**, horizontal scaling achieves the lowest normalized operating cost under the default P95 ≤ 1.0 s SLO on variable baseline traffic (1.9789 units, saving 6.48%–6.78% vs. hybrid and vertical, statistically significant), whereas vertical and hybrid reduce P95 latency by 15.0%–16.7%. Regarding **RQ2**, startup delays of 20–30 s cause horizontal scaling to violate the 1.0 s SLO (P95 = 3.78 s), but 1 warm standby instance (25% cost) cuts P95 latency by 79.4%, and a 60 s shutdown window eliminates inter-spike thrashing for +4.0%–6.3% cost. Regarding **RQ3**, hybrid scaling is significantly more cost-effective than horizontal scaling during sudden spikes (-1.40%, 95% CI [-0.0408, -0.0108]) and is the only paradigm that achieves 100% per-run compliance under strict tail-latency constraints (P95 ≤ 0.35 s) when single-server vertical capacity is capped at c_max,node = 2.

**Code and Data Availability:** The complete Python/SimPy simulation source code, 27-test verification suite, experimental CSV datasets, and plotting scripts are publicly available at https://github.com/SAS2005/Project-DTA400.

## REFERENCES


[1] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," *J. Grid Comput.*, vol. 12, no. 4, pp. 559–592, Dec. 2014.

[2] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, "Heterogeneity and dynamicity of clouds at scale: Google trace analysis," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, San Jose, CA, USA, 2012, pp. 1–13.

[3] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge, U.K.: Cambridge Univ. Press, 2013.

[4] W. Dawoud, I. Takouna, and C. Meinel, "Elastic VM for cloud resources provisioning optimization," in *Advances in Computing and Communications*, Berlin, Germany: Springer, 2011, pp. 431–445.

[5] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: A taxonomy and survey," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–33, Jul. 2018.

[6] N. R. Herbst, S. Kounev, and R. Reussner, "Elasticity in cloud computing: What it is, and what it is not," in *Proc. 10th Int. Conf. Autonomic Comput. (ICAC)*, San Jose, CA, USA, 2013, pp. 23–27.

[7] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley-Interscience, 1975.

[8] M. Mao and M. Humphrey, "A performance study on the VM startup time in the cloud," in *Proc. IEEE 5th Int. Conf. Cloud Comput. (CLOUD)*, Honolulu, HI, USA, 2012, pp. 423–430.

[9] P. Silva, C. Fireman, and T. E. Pereira, "Prebaking functions to warm the serverless cold start," in *Proc. 21st Int. Middleware Conf.*, Delft, Netherlands, 2020, pp. 1–13.

[10] N. Matloff, "Introduction to discrete-event simulation and the SimPy language," Dept. Comput. Sci., Univ. California, Davis, CA, USA, Tech. Rep., Feb. 2008.